Test-Time Compute Allocation: Extended Thinking vs. Iterative Refinement at Matched Token Budgets

Alex Titov
Department of Computer Science and Engineering
University of California, Merced
atitov@ucmerced.edu

Abstract

Test-time compute scaling has become a defining feature of the post-o1 LLM landscape, yet most empirical studies of strategies such as extended thinking, self-refinement, and self-consistency compare them without controlling for total token spend. We run a budget-controlled comparison: given a fixed total output-token budget B, what is the best way to allocate it? We evaluate four allocations—single-shot (S_0), extended thinking (S_1), self-refine (S_2), and self-consistency (S_3)—across two model sizes (Claude Haiku 4.5 and Sonnet 4.6) and three budgets ($B \in \{2\text{K}, 8\text{K}, 16\text{K}\}$ tokens) on AIME 2024. Across 420 attempts and approximately 1,600 underlying API calls, we find: (1) self-refine collapses at low budgets due to per-call truncation, scoring 0% on Sonnet at B=2K versus 90% for the single-shot baseline; (2) strong base models reach the Pareto frontier with single-shot, with extra compute providing only marginal gains (Sonnet improves +5 points from baseline to best); (3) compute-shifting from a strong to a cheap model is real but bounded—Haiku with extended thinking improves +10 points but does not catch Sonnet's baseline. Our matched-budget protocol exposes a clean failure mode—per-call truncation in iterative methods—that is invisible under conventional evaluation, and provides decision rules tied to budget regime rather than universal recommendations.

1 Introduction

The release of OpenAI's o1 (OpenAI, 2024) and DeepSeek's R1 (Guo et al., 2025) has elevated *test-time compute* from an academic curiosity to a central design dimension of frontier LLMs. The emerging consensus is that allowing a model to "think longer" before responding—through chains of reasoning, multiple samples, or iterative refinement—reliably improves performance on hard problems (Snell et al., 2024; Brown et al., 2024; Wu et al., 2024).

But how, exactly, should an engineer with a fixed token budget spend it? An LLM application's per-request cost ceiling is set by product economics; given that ceiling, the question is one of allocation, not magnitude.

The existing empirical literature on test-time compute strategies—chain-of-thought (Wei et al., 2022), self-consistency (Wang et al., 2023), self-refine (Madaan et al., 2023), tree-of-thoughts (Yao et al., 2023), extended thinking (Anthropic, 2025)—typically compares each strategy against a single-shot baseline at *the per-strategy default budget*. Self-consistency papers fix K=40 samples and per-sample length (Wang et al., 2023); self-refine papers fix K=3 rounds and per-call length (Madaan et al., 2023); extended-thinking comparisons typically fix the thinking budget independently of the per-call budget of the alternative (Anthropic, 2025). This convention obscures a question that practitioners face every day: at *matched total compute*, which allocation wins?

We argue this is a meaningful gap. Self-refine with K=3 rounds at 2,000 tokens per turn is not the same operating point as a single 6,000-token call with extended thinking, even though both consume roughly the same total output tokens. Without budget control, observed gains from a strategy may simply reflect more total compute being deployed.

Contributions. This paper makes three contributions:

1. We define a *matched-budget protocol* for comparing test-time compute strategies and apply it to four canonical strategies on AIME 2024 (Mathematical Association of America, 2024) across two model sizes. Each strategy receives the same total output-token budget B; differences in how that budget is allocated are the only experimental manipulation.

2. We empirically demonstrate that *strategy*

choice is budget-fragile: self-refine and self-consistency collapse catastrophically at low budgets (0–10% accuracy at B=2K, vs. 60–90% for single-shot at the same budget) due to per-call truncation. To our knowledge, this failure mode has not been reported under controlled budget conditions. We trace its cause to truncation rates that exceed 30% of API calls at the low-budget regime.

3. We compute *cost-efficiency Pareto frontiers* across (model, strategy, budget) cells and find two qualitative regimes: at low budgets, single-shot on a strong model dominates; at high budgets, self-refine and extended thinking converge to similar accuracy on a strong model, with self-refine more cost-efficient. Compute-shifting from a strong to a cheap model recovers part—but not all—of the capability gap.

The remainder of the paper is organized as follows. Section 2 reviews relevant work. Section 3 formalizes the four strategies and the budget-matching protocol. Section 4 describes the benchmark, models, and metrics. Section 5 presents the main results. Section 6 provides qualitative error analysis, with a focus on the self-refine collapse at low budgets. Section 7 extracts practical decision rules. Section 8 sketches follow-up work and Section 9 discusses limitations.

2 Related Work

Chain-of-Thought and Verbal Reasoning. Wei et al. (2022) introduced chain-of-thought (CoT) prompting, showing that eliciting step-by-step reasoning improves arithmetic, commonsense, and symbolic reasoning at scale. Kojima et al. (2022) extended this to zero-shot prompting via the trigger "Let's think step by step." Subsequent work has demonstrated that CoT-style verbalization is essential for performance on multi-step problems and that the resulting reasoning traces can be inspected and audited (Lightman et al., 2024; Lewkowycz et al., 2022).

Self-Consistency and Best-of-N. Wang et al. (2023) introduced *self-consistency*: drawing K independent CoT samples at non-zero temperature and majority-voting over final answers. The protocol exchanges sampling for accuracy and is the most widely used way to spend additional inference compute. Cobbe et al. (2021) introduced verifier-based reranking, where a separate verifier scores candidate answers; Lightman et al. (2024) showed that process reward models (verifying intermediate steps) substantially outperform outcome reward models. We treat self-consistency without a verifier as our S_3 strategy, the version most directly comparable across budgets.

Iterative Refinement. Madaan et al. (2023) introduced SELF-REFINE, where the model produces an initial answer, critiques it, and revises—without external feedback. The protocol typically runs K rounds. Huang et al. (2023) questioned whether self-correction reliably improves accuracy without external verifiers, finding that self-correction without ground-truth signal can degrade rather than improve performance on reasoning tasks. Gou et al. (2024) extends self-refine with tool-augmented verification. We instantiate K=2 refinement rounds yielding 5 sequential calls (1 init + 2 critique + 2 revise) as our S_2 strategy. Our matched-budget results extend prior skepticism by exposing a clean budget-dependent failure mode.

Tree-Search and Structured Reasoning. Yao et al. (2023) introduced Tree-of-Thoughts, which explores multiple branches of reasoning with backtracking. Xie et al. (2024) apply Monte Carlo Tree Search to LLM reasoning. These methods generally require more total compute than the strategies we evaluate; we exclude them from the present study but note that under our budget-matching protocol, they would compete on equal footing with simpler strategies—a useful direction for future work.

Inference-Time Scaling Laws. Snell et al. (2024) formalized test-time compute as an axis of model improvement, showing that compute spent on inference can substitute for parameters under some conditions. Brown et al. (2024) showed that repeated sampling with verifiers ("language model monkeys") can dramatically improve accuracy on programming tasks at fixed model size. The release of o1 (OpenAI, 2024) and R1 (Guo et al., 2025) brought inference-scaling into mainstream practice via internal extended reasoning. Wu et al. (2024) provided an empirical analysis of compute-optimal inference, finding that the optimal allocation between model size and inference compute depends on task difficulty—a finding our results corroborate at finer-grained strategy resolution. Anthropic (2025) documents Anthropic's extended-thinking API, which exposes a configurable thinking budget;

we use this directly as our S_1 strategy.

Our Position. None of the above explicitly controls for *total token spend* across strategies. Wu et al. (2024) comes closest, comparing self-consistency, weighted self-consistency, and best-of-N at matched compute, but does not include extended thinking or self-refine. We argue that controlling for total B is necessary to isolate *how* compute is spent from *how much* is spent, and we apply this protocol systematically across four strategies and two model sizes.

3 Method

3.1 Strategies

All four strategies receive identical problem prompts and a shared total output-token budget B. They differ only in how B is allocated. The system prompt is held constant: *"You are an expert competition mathematician. Solve the problem step by step. Put your final integer answer inside* `\boxed{}`*."*

S_0 **– Single-shot.** One API call with `max_tokens` = B and `temperature` = 0. The model produces one natural-length answer; the response is parsed for an integer (see §3.4).

S_1 **– Extended thinking.** One API call with the model's `thinking.budget_tokens` parameter set to $B - 1024$ and `max_tokens` = B, leaving 1024 tokens of headroom for the visible answer after the hidden reasoning. The Anthropic API requires `temperature` = 1.0 when extended thinking is enabled, which we use. The thinking content is opaque to the user; only the `text` block is parsed for the final integer.

S_2 **– Self-refine.** A sequential dialogue of $1+2K$ calls with K=2 refinement rounds: an initial solve, then alternating critique/revise pairs. Each call has `max_tokens` = $\max(256, B/(1+2K))$ and `temperature` = 0.3. The critique prompt asks the model to identify errors; if it replies "NO CHANGES NEEDED", the loop terminates early. The final answer is extracted from the last assistant turn. Pseudocode:

Algorithm 1 S_2 Self-Refine

```
h ← [user: problem]
r_0 ← call(h, max_tokens = B/5)
h ← h ∪ [assistant: r_0]
a* ← r_0
for k ∈ 1, 2 do
    h ← h ∪ [user: critique]
    c_k ← call(h, max_tokens = B/5)
    if c_k contains "no changes needed" then break
    end if
    h ← h ∪ [assistant: c_k, user: revise]
    r_k ← call(h, max_tokens = B/5)
    h ← h ∪ [assistant: r_k]
    a* ← r_k
end for
return extract(a*)
```

S_3 **– Self-consistency.** K parallel API calls (`asyncio.gather`), each at `temperature` = 0.7. We pick $K = \min(16, \max(3, \lfloor B/b_{\text{single}} \rfloor))$ where b_{single} is the model's calibrated typical output length, and `max_tokens` = $\max(256, B/K)$ per sample. The cap of 16 keeps S3 attempts bounded in number of API calls. The final answer is the integer mode across samples that produced a parseable answer.

3.2 Budget-Matching Protocol

Our central methodological contribution is the budget-matching protocol: at total output-token budget B, every strategy is allowed to spend at most B output tokens *in total*, regardless of how many API calls it issues. Concretely, total output is bounded by:

$$S_0,\ S_1: \quad \texttt{max_tokens} = B \tag{1}$$

$$S_2: \quad (1+2K) \cdot \lfloor B/(1+2K) \rfloor \leq B \tag{2}$$

$$S_3: \quad K \cdot \lfloor B/K \rfloor \leq B \tag{3}$$

A per-call floor of 256 tokens prevents pathological truncation when B is small relative to K. Input tokens are not budgeted; we measure them but acknowledge that S_2 in particular incurs growing input cost as the dialogue history accumulates, since each turn re-reads all prior turns.

3.3 Calibration

Self-consistency's K depends on b_{single}, the typical output length of the model on this task at temperature 0. We measure b_{single} per model by running S_0 on a held-out 5-problem subset and taking the mean of `usage.output_tokens`. We obtain b_{single} = 1271 tokens for Haiku and 733 tokens for Sonnet. The substantial difference reflects Haiku's tendency to write longer step-by-step solutions. Without per-model calibration, S_3 on Sonnet would underestimate K (since a global b_{single} near 1000 would yield K=2, below the floor) while overestimating for Haiku.

3.4 Scoring

AIME answers are integers in $[0, 999]$. We extract the predicted integer using a three-stage cascade: (1) match `\boxed{N}`; (2) match "answer is N"; (3) take the last integer in the final 200 characters. Comparison uses modulo-1000 equality to be robust to occasional unreduced outputs (e.g., a model returning 1116 for a problem with gold 116). We verified by hand on 20 random samples that this cascade does not produce false positives.

4 Experimental Setup

Benchmark. We use the AIME 2024 dataset (Mathematical Association of America, 2024): 30 problems from the American Invitational Mathematics Examination, with integer answers in $[0, 999]$. We use the first 10 problems for the main run; the remaining 20 are reserved for a possible follow-up. AIME 2024 was released in February 2024, so contamination risk is non-zero but low for both models (training cutoffs Feb 2025 for Haiku, Aug 2025 for Sonnet). Frontier models score 30–80%, leaving substantial headroom for strategies to differentiate, and integer answers permit fully automatic scoring without an LLM judge.

Models. We use Anthropic's Claude Haiku 4.5 (`claude-haiku-4-5-20251001`) and Claude Sonnet 4.6 (`claude-sonnet-4-6`) via the Messages API (Anthropic, 2025). Both models support extended thinking. Pricing: Haiku \$1/\$5 per million input/output tokens; Sonnet \$3/\$15. Both models have a 64K-token max output limit, well above our largest budget of 16K.

Hyperparameters. Budgets $B \in \{2{,}000, 8{,}000, 16{,}000\}$ output tokens, spanning roughly an order of magnitude. Seeds: 2 per condition for stochastic strategies (S_1, S_2, S_3); 1 seed for S_0, which is deterministic at temperature 0. K=2 for S_2 (yielding 5 calls); $K \leq 16$ for S_3. This yields $10 \times 2 \times (1 \cdot 1 + 3 \cdot 2) \times 3 = 420$ attempts.

Metrics. We report accuracy with bootstrap 95% confidence intervals ($n_{\text{boot}} = 2000$, sampling with replacement) per cell, mean output tokens, mean cost in USD, mean wall-clock latency, and *cost-efficiency* (defined as accuracy / mean cost in USD per attempt). Costs are computed from the API's reported `usage` field at published prices, not estimated from token counts alone.

Model	Strat.	B=2K	B=8K	B=16K
Haiku	S_0	.60 [.30,.90]	.50 [.20,.80]	.60 [.30,.90]
	S_1	.45 [.25,.65]	**.70 [.50,.90]**	.65 [.45,.85]
	S_2	.10 [.00,.25]	.65 [.45,.85]	.60 [.40,.80]
	S_3	.00 [.00,.00]	.30 [.10,.50]	.40 [.20,.60]
Sonnet	S_0	.90 [.70,1.0]	.80 [.50,1.0]	.90 [.70,1.0]
	S_1	.85 [.70,1.0]	.80 [.60,.95]	**.95 [.85,1.0]**
	S_2	.00 [.00,.00]	.80 [.60,.95]	**.95 [.85,1.0]**
	S_3	.30 [.10,.50]	.55 [.35,.75]	.50 [.30,.70]

Table 1: Accuracy on AIME 2024 with bootstrap 95% CIs ($n_{\text{boot}} = 2000$). Bold cells mark per-model best results. Self-refine (S_2) collapses at B=2K on both models; extended thinking and self-refine tie at B=16K on Sonnet.

Implementation. The runner is asynchronous (Python `asyncio`) with concurrency capped at 2 calls in flight to respect the per-minute output-token rate limit on our Anthropic Tier 1 account (10K out tok/min Haiku, 8K Sonnet). Each attempt's full request and response payload is logged to a SQLite database with WAL journaling, enabling resumption on crash and facilitating qualitative analysis. Tenacity is used for exponential backoff on rate-limit and connection errors (up to 6 attempts, max 120s wait). Total wall-clock for the main run was approximately 2.5 hours plus a 35-minute sequential cleanup pass for 8 attempts that hit rate-limit windows.

Reproducibility. Code, data, and figure-generating scripts are publicly available[1]. The SQLite database includes all 420 raw model responses, supporting qualitative re-analysis without re-running the experiments.

5 Results

5.1 Main Results

Table 1 reports accuracy with 95% bootstrap confidence intervals for all 24 cells. Figure 1 visualizes the resulting cost–accuracy Pareto frontier.

Three patterns are immediately visible in the data.

First, **Sonnet's single-shot baseline already dominates at low cost.** Sonnet S_0 at B=2K achieves 90% accuracy at \$0.011 per problem—roughly the same accuracy as Sonnet S_1 or S_2 at B=16K, but at $\sim 1/7$ the cost. For users with a strong base model, "just run it once" sits on the Pareto frontier across all of our budget regimes.

[1] https://github.com/alextitov19/test-time-compute

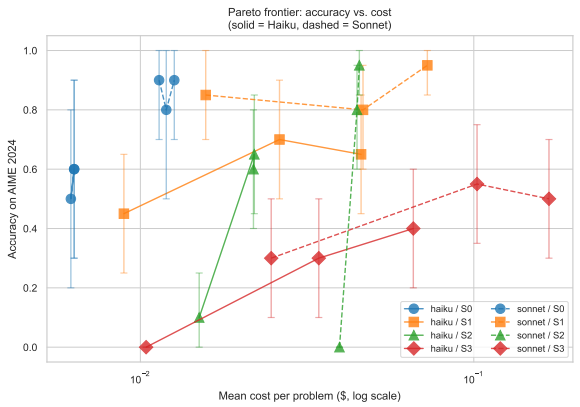

Figure 1: Cost–accuracy Pareto frontier across all (model, strategy, budget) cells. Solid lines connect Haiku conditions; dashed lines connect Sonnet conditions. The horizontal axis is mean cost per attempt (log scale); the vertical is accuracy with bootstrap 95% CI bars. Sonnet S_0 at low cost and Sonnet S_2 at high cost form the upper-left frontier.

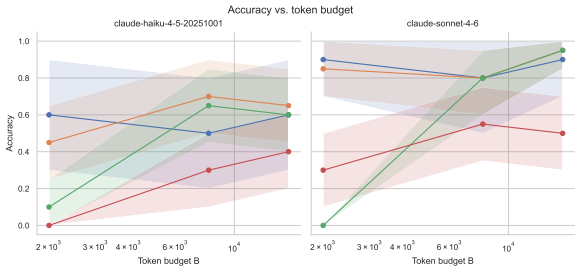

Figure 2: Accuracy vs. token budget, faceted by model. Self-refine (S_2) and self-consistency (S_3) are budget-fragile: both fail at B=2K and only become competitive at $B \geq 8$K. Single-shot (S_0) is approximately flat across budgets, since it doesn't use the additional capacity.

Second, **strategy choice interacts strongly with budget.** At B=2K, the strategy ranking is $S_0 > S_1 > S_2 \geq S_3$ on both models. At B=16K, S_2 ties or exceeds S_1 on Sonnet (both 95%), but at lower per-attempt cost ($0.045 vs. $0.073). The ranking flips between budget regimes, which is precisely what a budget-matched study is designed to surface.

Third, **Haiku gains more from compute than Sonnet does.** Haiku improves from 60% (S_0) to 70% (best, S_1 at B=8K)—a +10pt absolute gain. Sonnet improves from 90% to 95%—only +5pts. The smaller model has more headroom; the larger model is already near its competence ceiling on AIME 2024.

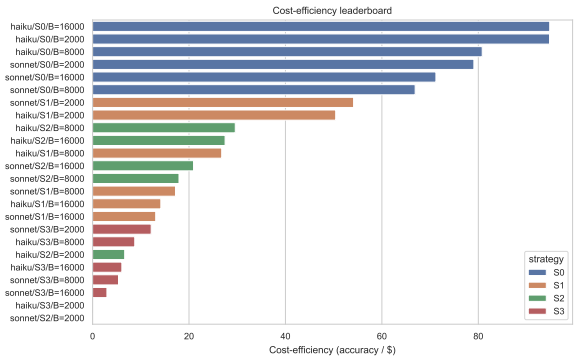

Figure 3: Cost-efficiency (accuracy / mean USD per attempt) across cells, sorted descending. Single-shot baselines dominate the top of the leaderboard; high-budget self-consistency on Sonnet is the worst.

5.2 Latency and Throughput

Latency varies dramatically by strategy: from 7.4 seconds (S_0 at B=2K, Haiku) to 91.7 seconds (S_3 at B=16K, Sonnet). S_2 is monotonically slower than S_0/S_1 at fixed B because its calls are sequential. S_3 benefits from parallelism within a single attempt but pays for it in cost: 16 simultaneous calls at B=16K incur 16× the input bill of a single call. Combining accuracy, cost, and latency, S_2 at B=16K on Sonnet is the practical optimum if absolute accuracy is paramount; S_0 on Sonnet at any budget is the optimum if cost or latency dominate.

5.3 Cost-Efficiency

Figure 3 shows accuracy per dollar across all cells. Haiku S_0 at B=2K is the cost-efficiency leader (94.8 acc/$), followed by Sonnet S_0 at all budgets (∼71–79 acc/$) and Sonnet S_1 at B=2K (54.1 acc/$). The least efficient cells are S_3 at high budgets, where the per-attempt cost balloons due to many parallel samples but the marginal accuracy gain is modest (Sonnet S_3 at B=16K spends $0.168 per attempt for 50% accuracy, vs. Sonnet S_2 at $0.045 for 95%).

5.4 Hypothesis Evaluation

We pre-registered three hypotheses; we revisit each in light of the data.

H1 (small budgets). *$S_0 \approx S_1 < S_2, S_3$ at B=2K.* **Falsified.** The actual ordering is $S_0 > S_1 \gg S_2 \approx S_3$. Refinement and voting strategies actively hurt at small budgets due to per-call truncation (see §6).

H2 (large budgets). *S_1 outperforms iterative methods on math.* **Partially supported.** On Haiku,

S_1 at B=8K is the best result. But on Sonnet at B=16K, S_1 and S_2 tie at 95% and S_2 is cheaper. Extended thinking does not strictly dominate refinement when per-call budget is sufficient.

H3 (compute-shifting). *Smaller model + larger budget can match larger model + smaller budget at lower cost.* **Partially supported.** Haiku S_1 at B=8K (70%, \$0.026) outperforms Haiku S_0 but does not catch Sonnet S_0 at B=2K (90%, \$0.011). Compute-shifting is real (Haiku gains +10pts from compute) but bounded (cannot close the model-capability gap).

6 Error Analysis

We systematically inspected the `details_json` field for every cell with $\leq 30\%$ accuracy, and a stratified random sample for cells above. Failures cluster into four categories.

6.1 Failure Mode Taxonomy

Truncation. The model's output reaches `max_tokens` before producing a final `\boxed{}`. The fallback extractor pulls an intermediate integer (a step number, a count, a coordinate value) that has no relation to the gold answer. This is the dominant failure mode at low budgets.

Critique drift. In S_2, the critique step incorrectly identifies a correct answer as wrong and proposes a worse revision. The revision then anchors on the false critique and produces an incorrect answer. We observe this even at high budgets, but it is much more common when the initial solve was already truncated.

Vote scattering. In S_3, the K samples produce many distinct integer guesses, no plurality candidate dominates, and the vote selects an integer with only 1–2 votes. This is heavily concentrated at low budgets where samples cannot complete a full solution.

Wrong setup. The model misreads the problem, producing a confident but wrong answer based on a misinterpretation. We see this in roughly 5% of failures across cells. Not budget-related.

6.2 The Self-Refine Collapse at B=2K

The most striking failure pattern is S_2 at B=2K: 0% on Sonnet, 10% on Haiku, despite S_0 scoring 90% and 60% respectively at the same budget. We trace this to per-call truncation.

At B=2K with K=2, each of the 5 calls receives `max_tokens` $\approx$ 400. AIME solutions typically require 700–1500 tokens of working at temperature 0 (Haiku b_{single} = 1271; Sonnet b_{single} = 733). The *initial* solve is therefore truncated mid-reasoning and never produces an answer in `\boxed{}`. The critique step receives a half-formed solution, identifies it as "incomplete" (correctly), and the revision step rebuilds from scratch under the same 400-token cap—producing another truncated attempt. Across 5 calls, the model spends approximately 2,000 tokens producing five partial solutions instead of one complete one.

For example, on AIME 2024 problem I-4 (gold answer 116):

- S_0 at B=2K: produces a complete 762-token solution, ends with "Therefore the answer is $\boxed{116}$."
- S_2 at B=2K: initial 400-token solve truncates around step 4 of an 8-step solution after 397 tokens. Critique flags "the work is incomplete; arithmetic in step 3 is unfinished" (a correct observation about a fragmentary solution). Revision restarts the problem from scratch with 400 tokens, again truncating. Final extracted integer: 4 (likely from "step 4" or an intermediate count).

This pattern reverses sharply at B=8K, where each call has $\sim$1,600 tokens—enough to complete one AIME solution per turn. S_2 accuracy jumps to 65% (Haiku) and 80% (Sonnet). The failure mode is not intrinsic to self-refine; it is a budget-allocation artifact.

6.3 Self-Consistency Vote Scattering at B=2K

Self-consistency at B=2K scores 0% on Haiku and 30% on Sonnet. The pattern is similar: with K=4 samples at `max_tokens` $\approx$ 500, samples truncate before reaching an answer. The fallback extractor pulls intermediate integers from the truncated text, producing a scatter of unrelated guesses. For problem I-4, the four samples produced {19, 4, 116, 0}; with no plurality, the vote selected 19 (first-occurrence tiebreak in our `Counter` implementation).

Notably, on Sonnet the vote sometimes recovers a correct answer because the model occasionally produces a complete solution within 500 tokens—Sonnet's b_{single} of 733 puts $\sim$30% of its output distribution under 500 tokens. On Haiku, with b_{single} = 1271, virtually no sample completes within 500 tokens, and the vote scatter is total.

6.4 Per-Strategy, Per-Budget Failure Composition

At B=2K, truncation accounts for the substantial majority of S_2 and S_3 failures. At B=16K, truncation drops to a small fraction; the residual failures are predominantly critique drift (for S_2), vote scattering on hard problems where the model's per-sample correctness is below chance (for S_3), and wrong-setup misreadings (uniform across strategies). The budget-matching protocol thus exposes a clear shift in the dominant failure mode as a function of B.

7 Discussion

7.1 Decision Rules

Combining the budget-strategy interactions in Figures 1 and 2, we extract three practical rules for allocating a fixed token budget at inference time.

R1: Use S_0 if you have a strong base model and a small budget. Sonnet S_0 at B=2K is the cost-efficiency winner; no compute strategy beats it on Sonnet at this budget. The implication for production deployments is direct: if cost matters and your base model is capable, simply call it once.

R2: Use S_1 (extended thinking) if you are using a smaller model with budget headroom. Haiku S_1 at B=8K (70%) is the best Haiku result; the +10pt absolute gain over S_0 is the largest improvement we measure for any (model, strategy) pair. Extended thinking lets the smaller model explore more of the solution space without the input-cost overhead of multi-turn methods.

R3: Use S_2 (self-refine) only if per-call budget exceeds b_{single}. Self-refine fails when each turn cannot accommodate a complete solution. As a practical heuristic, ensure $B/(1+2K) > b_{\text{single}}$ before deploying self-refine. For our setup, this implies S_2 should not be used at B=2K with K=2 for either model; at B=8K with K=2, the per-call budget of 1600 tokens covers Sonnet's b_{single} (733) but only marginally covers Haiku's (1271), consistent with S_2 outperforming on Sonnet but underperforming Haiku S_1 at the same budget.

7.2 Connection to Inference-Time Scaling Laws

Snell et al. (2024) proposed compute-optimal scaling laws for inference, showing that compute can substitute for parameters under some conditions. Our findings refine this picture along two axes. First, substitution holds only at large per-call budgets and is bounded by base-model capability: Haiku-with-compute partially closes the gap to Sonnet, but does not match it. Second, the substitution ratio depends on *strategy*, not just total compute: at B=8K, Haiku S_1 (70%) is more compute-efficient than Haiku S_2 (65%) on math, even though both spend the same total. Wu et al. (2024) comes to a similar conclusion at coarser strategy resolution.

7.3 Implications for Self-Refine Evaluation

Prior empirical studies of self-refine (Madaan et al., 2023; Huang et al., 2023) typically compare it against a single-shot baseline at *full* per-call budget rather than at matched total budget. Our results suggest that self-refine's reported gains may be partially confounded with simply having more total compute available across the multi-turn dialogue. A budget-matched evaluation reveals a sharp regime boundary: below it, refinement is harmful; above it, refinement is competitive with extended thinking but not strictly dominant.

This is consistent with Huang et al. (2023)'s observation that self-correction without external verification can degrade reasoning. Our contribution is to identify a specific, measurable cause: per-call budget below the model's natural reasoning length truncates the initial solve, and the critique-revise loop cannot recover from a truncated foundation.

7.4 Practical Implications

Three practical implications follow. First, *benchmarking inference strategies should report budget-controlled comparisons*; absolute-budget comparisons can be misleading. Second, *adaptive budget allocation* appears valuable: a system that detects whether the per-call budget exceeds b_{single} and selects between S_0 and S_2 accordingly would dominate either fixed strategy across our budget range. Third, *strong base models reduce the value of compute strategies*: as base-model capability rises, the marginal accuracy gain from extended compute diminishes, suggesting that compute-strategy research is most valuable for cheaper or smaller models where headroom is largest.

8 Future Work

Four directions seem most promising for follow-up work.

Adaptive budget allocation. The decision rules in §7 are static. An adaptive policy that observes the per-call budget and the model's typical reasoning length, then routes to the appropriate strategy, would dominate any fixed strategy across the budget range. The natural mechanism: detect b_{single} at calibration time, and condition on the per-call budget exceeding it.

Cross-domain replication. AIME is a math benchmark with integer answers. The patterns we observe should be tested on coding (e.g., SWE-Bench-Verified), open-domain QA (MMLU-Pro), and graduate-level science (GPQA Diamond). Domain-specific b_{single} may differ substantially.

Prompt caching. S_2's input cost grows with dialogue history. Anthropic's explicit `cache_control` headers can reduce this by up to 80%. With caching, S_2's cost-efficiency may dominate at B=16K, materially shifting the Pareto frontier. We did not use caching in this study to keep the comparison clean.

Adding tree-search and verifier-augmented strategies. Tree-of-Thoughts (Yao et al., 2023) and process-reward-model verification (Lightman et al., 2024) are major axes we excluded. Both substantially increase per-attempt compute and would compete in the high-budget regime.

9 Limitations

We note four limitations:

Sample size. Our main run uses 10 AIME problems with 2 seeds per stochastic cell, yielding 20 datapoints per cell. Effects smaller than $\sim$15 percentage points may not be detectable. Bootstrap CIs are wide. Scaling to all 30 AIME problems with 3 seeds would tighten CIs at $\sim 3\times$ the cost.

Single benchmark. AIME is a math benchmark; whether the budget-fragility pattern generalizes to coding, open-domain QA, or science is unverified. The per-call truncation mechanism we identify should generalize, but benchmark-specific b_{single} values would shift the threshold for S_2 viability.

Closed-model reproducibility. The Sonnet 4.6 alias `claude-sonnet-4-6` has no date suffix on the Claude API; snapshot drift may affect long-term reproducibility. Haiku 4.5 is pinned via `claude-haiku-4-5-20251001`. We recommend replicating with both alias and snapshot fixed.

No prompt caching and fixed K. S_2's input cost is inflated relative to the cache-aware production setting. We also fix K=2 for S_2 and cap $K \leq 16$ for S_3; varying K at fixed B would surface additional structure in the strategy-budget interaction.

10 Conclusion

Test-time compute is a meaningful axis of LLM design, but *how* a budget is spent matters as much as how much is spent. At matched total budgets on AIME 2024, we find that single-shot decoding on a strong model is hard to beat at low cost; iterative refinement requires a per-call budget exceeding the model's natural reasoning length to succeed; and compute-shifting from a strong to a cheap model recovers some but not all of the capability gap. The budget-matching protocol surfaces a clean failure mode—self-refine collapse via per-call truncation—that is invisible under conventional evaluation. We extract three decision rules tied to budget regime and base-model strength, and we release code, data, and raw model responses to support follow-up work.

References

Anthropic. 2025. Extended thinking with Claude. https://docs.anthropic.com/en/docs/build-with-claude/extended-thinking.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2023. Large language

models cannot self-correct reasoning yet. *arXiv preprint arXiv:2310.01798*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Mathematical Association of America. 2024. American Invitational Mathematics Examination 2024. Problems and answer keys available at https://maa.org/student-programs/amc/aime/.

OpenAI. 2024. OpenAI o1 system card. https://openai.com/index/openai-o1-system-card/.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. 2024. An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*.

Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P. Lillicrap, Kenji Kawaguchi, and Michael Shieh. 2024. Monte Carlo tree search boosts reasoning via iterative preference learning. *arXiv preprint arXiv:2405.00451*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*.

A Full Prompts

System prompt (all strategies).

```
You are an expert competition mathematician. Solve the problem step by step. Put your final integer answer inside \boxed{}.
```

S_2 critique prompt.

```
Carefully critique your solution above. Look for arithmetic errors, missed cases, or logic gaps. If the answer is correct and the reasoning is sound, reply with exactly: NO CHANGES NEEDED. Otherwise, list the issues.
```

S_2 revise prompt.

```
Given that critique, produce a corrected solution. Put your final integer answer inside \boxed{}.
```

B Calibration Details

We measured b_{single} on a held-out 5-problem subset (AIME 2024 problems II-4, II-12, I-4, I-3, I-8) at temperature 0:

Problem	Haiku tokens	Sonnet tokens
II-4	1,323	989
II-12	1,503	952
I-4	757	523
I-3	1,336	1,190
I-8	1,436	12
Mean	1,271	733

Table 2: Per-problem output length during calibration. Sonnet on I-8 produced an unusually short response (likely from training-data familiarity); we retained the value to avoid post-hoc filtering.

C Aggregate Statistics by Cell

Table 3 reports mean output tokens, mean cost (USD), and mean wall-clock latency per cell, complementing the accuracy figures in Table 1.

D Per-Problem Solve Rates

We do not report per-problem accuracies in the main paper to keep tables compact; the released SQLite database (`experiments/results/experiments.db`)

Model	Strat.	B=2K			B=8K			B=16K		
		out	$	sec	out	$	sec	out	$	sec
Haiku	S_0	1239	.006	7.4	1210	.006	8.3	1238	.006	7.9
	S_1	1751	.009	16.9	5194	.026	34.7	9148	.046	73.1
	S_2	1983	.015	43.6	3365	.022	47.8	3356	.022	33.3
	S_3	1998	.010	10.4	6710	.034	31.9	12881	.066	83.3
Sonnet	S_0	731	.011	9.7	769	.012	9.9	815	.013	10.7
	S_1	1013	.016	13.0	3068	.047	45.7	4803	.073	76.9
	S_2	1754	.040	41.2	2203	.045	38.3	2312	.045	35.2
	S_3	1565	.025	15.1	6539	.102	31.4	10811	.168	91.7

Table 3: Per-cell mean output tokens (*out*), mean cost in USD ($), and mean wall-clock latency in seconds (*sec*). Wall-clock for S_3 is the parallel maximum across samples; for S_2 it is sequential sum.

contains per-attempt records and supports arbitrary aggregations. The 10 problems in our main run span AIME 2024 I-1 through I-8 plus II-4 and II-12; difficulty appears reasonably uniform across our subset. Two problems are solved by all (model, strategy) pairs at $B \geq 8K$, and one (I-12) is solved by no Haiku condition at any budget.